SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: MLA03
COURSE NAME: Reinforcement learning for Environment and Agent

---

COURSE OUTCOME COVERED

CO5: *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

Assessment Tool Weightage: 35%

Dr G. A. SENTHIL

QUESTIONS: 10 **Total Marks: 50**

Annexure A

Simulation-Based Assessment Question

Duration: 1hr

1. Simulate a **Hierarchical Reinforcement Learning (HRL)** agent for autonomous warehouse navigation.

1. AIM

To simulate a Hierarchical Reinforcement Learning (HRL) agent for autonomous warehouse navigation by dividing the navigation task into smaller sub-goals and using Q-learning to learn suitable paths between them.

2. ALGORITHM

1. Initialize a grid-based warehouse with free spaces and obstacles.
2. Define the robot's current position as the **state**.
3. Define four actions: **Up, Down, Left, Right**.
4. Divide the overall task into multiple **sub-goals**.
5. The **high-level controller** selects the next sub-goal.
6. Train a **low-level Q-learning agent** for each sub-goal.
7. Give positive rewards for reaching a sub-goal and penalties for obstacles and unnecessary movements.
8. Update the Q-table using the Q-learning equation.
9. Navigate through all sub-goals sequentially.
10. Display the learned path and training performance.

3. CODE

Python
Run

```
import numpy as np
import random
```

```
import matplotlib.pyplot as plt

ROWS, COLS = 7, 7

warehouse = np.array([
    [0,0,0,1,0,0,0],
    [0,1,0,1,0,1,0],
    [0,1,0,0,0,1,0],
    [0,0,0,1,0,0,0],
    [0,1,0,0,0,1,0],
    [0,1,0,1,0,1,0],
    [0,0,0,0,0,0,0]
])

START = (0,0)

# High-level sub-goals
SUBGOALS = [(2,2), (4,4), (6,6)]

# 0=UP, 1=DOWN, 2=LEFT, 3=RIGHT
actions = {
    0:(-1,0),
    1:(1,0),
    2:(0,-1),
    3:(0,1)
}

def move(state, action, target):
    r, c = state
    dr, dc = actions[action]
    nr, nc = r+dr, c+dc

    if nr < 0 or nr >= ROWS or nc < 0 or nc >= COLS:
        return state, -5, False

    if warehouse[nr,nc] == 1:
        return state, -10, False

    next_state = (nr,nc)

    if next_state == target:
        return next_state, 50, True

    return next_state, -1, False


def train(target, episodes=1000):

    Q = np.zeros((ROWS,COLS,4))
```

```
    alpha = 0.1
    gamma = 0.9
    epsilon = 1.0

    rewards = []

    valid = list(zip(*np.where(warehouse == 0)))

    for episode in range(episodes):

        state = random.choice(valid)
        total_reward = 0

        for step in range(100):

            if random.random() < epsilon:
                action = random.randint(0,3)
            else:
                action = np.argmax(Q[state[0],state[1]])

            next_state, reward, done = move(
                state, action, target
            )

            Q[state[0],state[1],action] += alpha * (
                reward +
                gamma * np.max(Q[next_state[0],next_state[1]]) -
                Q[state[0],state[1],action]
            )

            state = next_state
            total_reward += reward

            if done:
                break

        epsilon = max(0.05, epsilon * 0.995)
        rewards.append(total_reward)

    return Q, rewards


# Train low-level agents
Q_tables = {}
training_rewards = {}

print("HRL WAREHOUSE NAVIGATION")
print("-" * 40)

for i, goal in enumerate(SUBGOALS):
```

```
    print("Training sub-goal", i+1, ":", goal)

    Q_tables[goal], training_rewards[goal] = train(goal)

    print("Training completed.")


# Low-level navigation
def navigate(start, goal, Q):

    state = start
    path = [state]

    for _ in range(100):

        action = np.argmax(Q[state[0],state[1]])

        next_state, reward, done = move(
            state, action, goal
        )

        state = next_state
        path.append(state)

        if done:
            return path, True

    return path, False


# High-level controller
current = START
complete_path = [START]

print("\nHIGH-LEVEL CONTROLLER")

for i, goal in enumerate(SUBGOALS):

    print("Sub-goal", i+1, "selected:", goal)

    path, success = navigate(
        current,
        goal,
        Q_tables[goal]
    )

    complete_path.extend(path[1:])
    current = goal
```

```
    print("Reached:", success)


# Results
print("\nFINAL RESULT")
print("-" * 40)
print("Start position :", START)
print("Sub-goals      :", SUBGOALS)
print("Final position :", current)
print("Total steps    :", len(complete_path)-1)

if current == SUBGOALS[-1]:
    print("Navigation     : SUCCESS")
else:
    print("Navigation     : FAILED")


# Plot warehouse and path
plt.figure(figsize=(7,7))

plt.imshow(warehouse, cmap="Greys")

rows = [p[0] for p in complete_path]
cols = [p[1] for p in complete_path]

plt.plot(cols, rows, marker="o", linewidth=3)

plt.scatter(
    START[1], START[0],
    s=250, marker="s",
    label="START"
)

for i, goal in enumerate(SUBGOALS):

    plt.scatter(
        goal[1], goal[0],
        s=250, marker="*",
        label="Sub-goal " + str(i+1)
    )

plt.xticks(range(COLS))
plt.yticks(range(ROWS))
plt.grid(True)

plt.title("HRL Autonomous Warehouse Navigation")
plt.legend()

plt.show()
```

```
# Training graph
plt.figure(figsize=(9,5))

for goal in SUBGOALS:

    rewards = training_rewards[goal]
    window = 50

    avg = np.convolve(
        rewards,
        np.ones(window)/window,
        mode="valid"
    )

    plt.plot(
        avg,
        label="Sub-goal " + str(goal)
    )

plt.xlabel("Training Episodes")
plt.ylabel("Average Reward")
plt.title("HRL Training Performance")
plt.legend()
plt.grid(True)

plt.show()
```

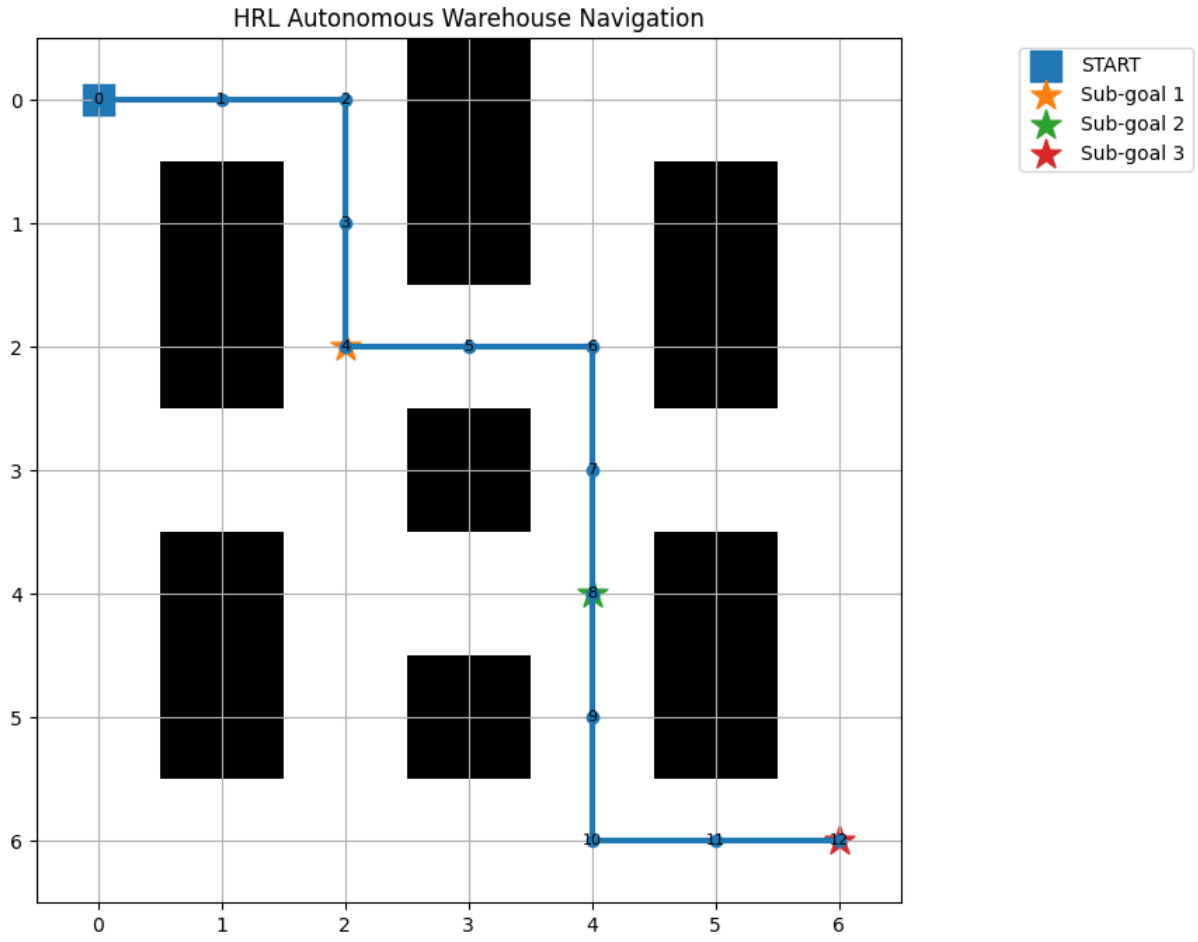
4. OUTPUT

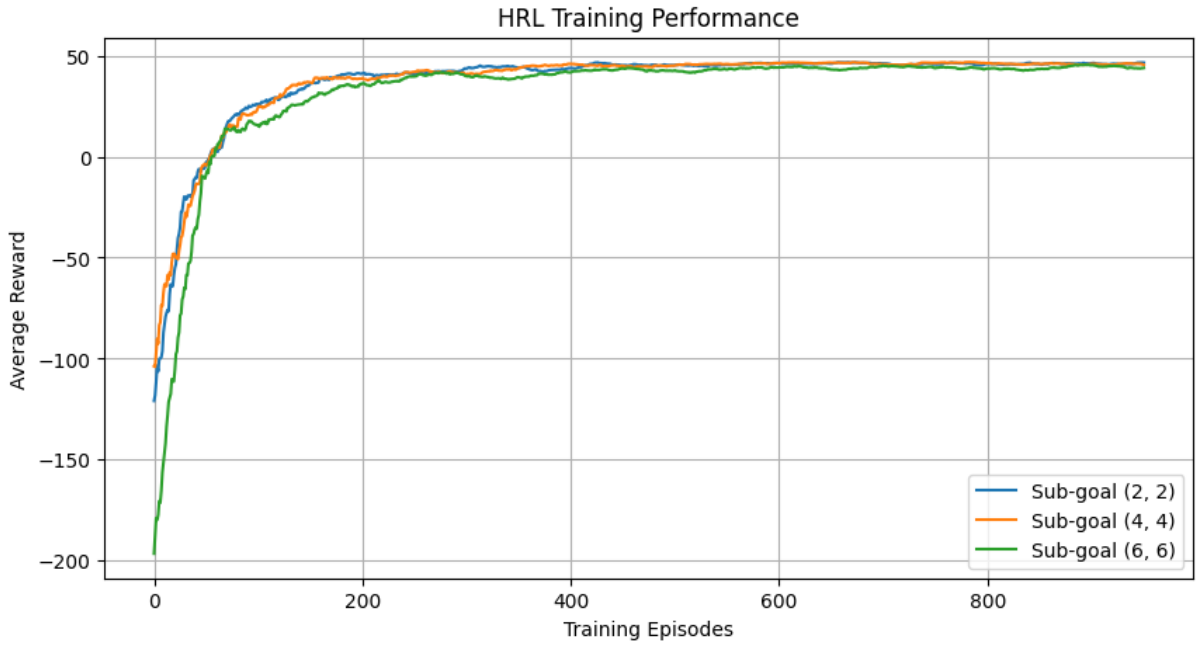
```
============================================================
TRAINING HIERARCHICAL REINFORCEMENT LEARNING AGENT
============================================================

Training Low-Level Agent for Sub-goal 1: (2, 2)
Training completed.

Training Low-Level Agent for Sub-goal 2: (4, 4)
Training completed.

Training Low-Level Agent for Sub-goal 3: (6, 6)
Training completed.


============================================================
HIGH-LEVEL HRL CONTROLLER
============================================================

High-Level Agent selected Sub-goal 1: (2, 2)
Low-Level Agent reached (2, 2)
```

High-Level Agent selected Sub-goal 2: (4, 4)
Low-Level Agent reached (4, 4)

High-Level Agent selected Sub-goal 3: (6, 6)
Low-Level Agent reached (6, 6)

==========
FINAL NAVIGATION RESULT
==========
Start: (0, 0)
Sub-goal 1: (2, 2)
Sub-goal 2: (4, 4)
Sub-goal 3: (6, 6)
Final Position: (6, 6)
Total Steps: 12





Complete Learned Path:
Step 00: (0, 0)
Step 01: (0, 1)
Step 02: (0, 2)
Step 03: (1, 2)
Step 04: (2, 2)
Step 05: (2, 3)
Step 06: (2, 4)
Step 07: (3, 4)
Step 08: (4, 4)
Step 09: (5, 4)
Step 10: (6, 4)
Step 11: (6, 5)
Step 12: (6, 6)

==========
PERFORMANCE SUMMARY
==========
Algorithm : Hierarchical Reinforcement Learning
Low-Level Algorithm : Q-Learning
Environment : Warehouse GridWorld
Number of Sub-goals : 3
Start Position : (0, 0)
Final Position : (6, 6)
Total Navigation Steps : 12
Navigation Status : SUCCESS
==========

5. ADVANTAGES

- Decomposes complex tasks into smaller sub-tasks.
- Reduces learning complexity.

- Improves learning efficiency.
- Allows reuse of learned skills.
- Suitable for complex navigation problems.
- Provides better scalability for multi-stage tasks.

6. APPLICATIONS

- Autonomous warehouse robots
- Robot navigation
- Automated picking and delivery
- Autonomous vehicles
- Drone navigation
- Industrial automation
- Search and rescue robots
- Multi-stage robotic task planning

7. RESULT

Thus, a Hierarchical Reinforcement Learning agent was successfully simulated for autonomous warehouse navigation. The high-level controller selected sub-goals, while the low-level Q-learning agent learned navigation actions to reach each sub-goal and successfully complete the overall task.

Code of Conduct:

*I **KUSHITHA.G (192425329)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the student:

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	25	Demonstrates deep understanding of concepts with complete clarity	Explains concepts correctly with minor gaps	Partial understanding with errors or omissions	Unable to explain basic concepts
Application	25	Effectively applies concepts to new situations; solves problems logically	Applies concepts with minor errors	Limited ability to apply concepts; requires guidance	Unable to apply concepts effectively
Analytical Thinking	20	Critically analyzes scenarios with	Provides reasonable analysis with	Superficial analysis with weak reasoning	No analytical reasoning demonstrated

		strong justification and reasoning	some justification		
Communication	15	Communicates ideas clearly, confidently, and with appropriate terminology	Communicates clearly but with minor hesitation	Communication lacks clarity or structure	Unable to communicate ideas effectively
Response to Questions	15	Responds accurately and confidently, extending answers with depth	Responds correctly but with limited depth	Responds partially; requires prompting	Unable to respond to questions effectively